MongoDB Complete Commands

Consider the following RDBMS table and insert into mongodb

eid	name	dob	doj	designation	salary	skills	address	accessid
101	Arjun Patel	1985-04-12	2010-03-15	Senior Developer	90000	["Java", "SQL", "Spring"]	{"street": "123 Elm St", "city": "New York", "state": "NY"}	[1234, 5678, 9101]
102	Meera Gupta	1990-08-05	2012-07-22	Project Manager	95000	["Leadership", "Agile", "Scrum"]	{"street": "456 Maple St", "city": "San Francisco", "state": "CA"}	[2345, 6789, 1012, 3456]
103	Rahul Krishna	1982-11-15	2008-02-10	UX Designer	87000	["Adobe XD", "Figma", "Sketch"]	{"street": "789 Oak St", "city": "Dallas", "state": "TX"}	[3456, 7890, 1234, 5678, 9012]
104	Sita Reddy	1975-01-30	2005-09-19	Network Engineer	92000	["Cisco Networks", "Security"]	{"street": "321 Pine St", "city": "Las Vegas", "state": "NV"}	[4567, 8901]
105	Lakshmi Narayan	1988-03-24	2013-08-01	Data Analyst	88000	["Python", "R", "Excel"]	{"street": "654 Spruce St", "city": "Boston", "state": "MA"}	[5678, 9012, 3456, 7890]
106	Dev Sharma	1992-12-11	2015-06-01	Software Tester	85000	["Selenium", "QA Tools", "Automation"]	{"street": "987 Cedar St", "city": "Miami", "state": "FL"}	[6789, 1012, 2345]

Create Database:

```
use employeeDB
```

Create Collection:

```
db.createCollection("employees")
```

Insert Documents:

```
db.employees.insertMany([
 {
  eid: 101,
  name: "Arjun Patel",
  dob: "1985-04-12",
  doj: "2010-03-15",
  designation: "Senior Developer",
  salary: 90000,
  skills: ["Java", "SQL", "Spring"],
  address: {
   street: "123 Elm St",
   city: "New York",
   state: "NY"
  },
  accessid: [1234, 5678, 9101]
 },
 {
  eid: 102,
  name: "Meera Gupta",
  dob: "1990-08-05",
  doj: "2012-07-22",
  designation: "Project Manager",
  salary: 95000,
  skills: ["Leadership", "Agile", "Scrum"],
  address: {
   street: "456 Maple St",
   city: "San Francisco",
   state: "CA"
  },
  accessid: [2345, 6789, 1012, 3456]
 },
 {
  eid: 103,
  name: "Rahul Krishna",
  dob: "1982-11-15",
  doj: "2008-02-10",
  designation: "UX Designer",
  salary: 87000,
```

```
  skills: ["Adobe XD", "Figma", "Sketch"],
  address: {
    street: "789 Oak St",
    city: "Dallas",
    state: "TX"
  },
  accessid: [3456, 7890, 1234, 5678, 9012]
},
{
  eid: 104,
  name: "Sita Reddy",
  dob: "1975-01-30",
  doj: "2005-09-19",
  designation: "Network Engineer",
  salary: 92000,
  skills: ["Cisco Networks", "Security"],
  address: {
    street: "321 Pine St",
    city: "Las Vegas",
    state: "NV"
  },
  accessid: [4567, 8901]
},
{
  eid: 105,
  name: "Lakshmi Narayan",
  dob: "1988-03-24",
  doj: "2013-08-01",
  designation: "Data Analyst",
  salary: 88000,
  skills: ["Python", "R", "Excel"],
  address: {
    street: "654 Spruce St",
    city: "Boston",
    state: "MA"
  },
  accessid: [5678, 9012, 3456, 7890]
},
{
  eid: 106,
  name: "Dev Sharma",
  dob: "1992-12-11",
  doj: "2015-06-01",
  designation: "Software Tester",
  salary: 85000,
  skills: ["Selenium", "QA Tools", "Automation"],
  address: {
    street: "987 Cedar St",
```

```
      city: "Miami",
      state: "FL"
    },
    accessid: [6789, 1012, 2345]
  }
])
```

Insert new values:

```
db.employees.insertOne({
  eid: 107,
  name: "Vikram Aditya",
  dob: "1993-05-21",
  doj: "2018-01-10",
  designation: "Cloud Architect",
  salary: 98000,
  skills: ["AWS", "Azure", "Docker"],
  address: {
    street: "201 Rose St",
    city: "Seattle",
    state: "WA"
  },
  accessid: [4321, 8765, 3210]
})
```

Retrieve values (only Projections)

Example	Command	Description
1	db.employees.find({}, { name: 1, designation: 1, _id: 0 })	Retrieve only the name and designation fields for all employees, excluding the _id field.
2	db.employees.find({}, { accessid: 0, address: 0 })	Retrieve all details for employees, but exclude the accessid and address fields.
3	db.employees.find({}, { salary: 0, skills: 0, _id: 1 })	Retrieve all fields except salary and skills, including _id (as it is included by default).

MongoDB Comparison Operators with Projections

Operator	Command	Description	Output
$eq	db.employees.find({ eid: { $eq: 102 } }, { eid: 1, name: 1, _id: 0 })	Finds employees where eid equals 102, showing only eid and name.	[{ eid: 102, name: "Meera Gupta" }]
$ne	db.employees.find({ designation: { $ne: "Project Manager" } }, { eid: 1, name: 1, _id: 0 })	Finds employees where designation is **not** "Project Manager", showing only eid and name.	[{ eid: 101, name: "Arjun Patel" }, { eid: 103, name: "Rahul Krishna" }, { eid: 104, name: "Sita Reddy" }]
$gt	db.employees.find({ salary: { $gt: 90000 } }, { eid: 1, name: 1, _id: 0 })	Finds employees where salary is **greater than** 90000, showing only eid and name.	[{ eid: 102, name: "Meera Gupta" }]
$gte	db.employees.find({ salary: { $gte: 90000 } }, { eid: 1, name: 1, _id: 0 })	Finds employees where salary is **greater than or equal to** 90000, showing only eid and name.	[{ eid: 101, name: "Arjun Patel" }, { eid: 102, name: "Meera Gupta" }, { eid: 104, name: "Sita Reddy" }]

Operator	Command	Description	Output
$lt	db.employees.find({ salary: { $lt: 90000 } }, { eid: 1, name: 1, _id: 0 })	Finds employees where salary is **less than** 90000, showing only eid and name.	[{ eid: 103, name: "Rahul Krishna" }, { eid: 105, name: "Lakshmi Narayan" }, { eid: 106, name: "Dev Sharma" }]
$lte	db.employees.find({ salary: { $lte: 90000 } }, { eid: 1, name: 1, _id: 0 })	Finds employees where salary is **less than or equal to** 90000, showing only eid and name.	[{ eid: 101, name: "Arjun Patel" }, { eid: 103, name: "Rahul Krishna" }, { eid: 106, name: "Dev Sharma" }]
$in	db.employees.find({ eid: { $in: [101, 103, 105] } }, { eid: 1, name: 1, _id: 0 })	Finds employees where eid is **in** the specified array [101, 103, 105], showing only eid and name.	[{ eid: 101, name: "Arjun Patel" }, { eid: 103, name: "Rahul Krishna" }, { eid: 105, name: "Lakshmi Narayan" }]
$nin	db.employees.find({ eid: { $nin: [101, 102] } }, { eid: 1, name: 1, _id: 0 })	Finds employees where eid is **not in** the specified array [101, 102], showing only eid and name.	[{ eid: 103, name: "Rahul Krishna" }, { eid: 104, name: "Sita Reddy" }, { eid: 106, name: "Dev Sharma" }]

MongoDB Logical Operators:

Operator	Command	Description	Output
$and	db.employees.find({ $and: [{ salary: { $gt: 80000 } }, { designation: "Senior Developer" }] }, { eid: 1, _id: 0 })	Finds employees where salary is greater than 80000 **and** the designation is "Senior Developer", showing only eid.	[{ eid: 101 }]
$or	db.employees.find({ $or: [{ salary: { $gt: 90000 } }, { designation: "Data Analyst" }] }, { eid: 1, _id: 0 })	Finds employees where salary is greater than 90000 **or** the designation is "Data Analyst", showing only eid.	[{ eid: 102 }, { eid: 105 }]

Operator	Command	Description	Output
$not	db.employees.find({ salary: { $not: { $gte: 90000 } } }, { eid: 1, _id: 0 })	Finds employees where salary is **not** greater than or equal to 90000, showing only eid.	[{ eid: 103 }, { eid: 105 }, { eid: 106 }]
$nor	db.employees.find({ $nor: [{ salary: { $gt: 90000 } }, { designation: "Data Analyst" }] }, { eid: 1, _id: 0 })	Finds employees where salary is **not greater than** 90000 **nor** the designation is "Data Analyst", showing only eid.	[{ eid: 103 }, { eid: 104 }, { eid: 106 }]

MongoDB Element Operators:

Operator	Command	Description	Output
$exists	db.employees.find({ skills: { $exists: true } }, { eid: 1, _id: 0 })	Finds employees where the skills field **exists**, showing only eid.	[{ eid: 101 }, { eid: 102 }, { eid: 103 }, { eid: 104 }, { eid: 105 }, { eid: 106 }]
$type	db.employees.find({ salary: { $type: "number" } }, { eid: 1, _id: 0 })	Finds employees where the salary field is of type number, showing only eid.	[{ eid: 101 }, { eid: 102 }, { eid: 103 }, { eid: 104 }, { eid: 105 }, { eid: 106 }]

MongoDB Evaluation Operators:

Operator	Command	Description	Output
$regex	db.employees.find({ name: { $regex: /^A/ } }, { eid: 1, _id: 0 })	Finds employees where the name field starts with "A" using a regular expression.	[{ eid: 101 }]

Operator	Command	Description	Output
$text	db.employees.find({ $text: { $search: "Developer" } }, { eid: 1, _id: 0 })	Finds employees where the designation field contains the word "Developer" (requires a text index).	[{ eid: 101 }]
$where	db.employees.find({ $where: "this.salary > 90000" }, { eid: 1, _id: 0 })	Finds employees where the salary is greater than 90000 using JavaScript expression.	[{ eid: 102 }, { eid: 104 }]
$expr	db.employees.find({ $expr: { $gt: ["$salary", 90000] } }, { eid: 1, _id: 0 })	Finds employees where the salary is greater than 90000 using an expression.	[{ eid: 102 }, { eid: 104 }]
$mod	db.employees.find({ eid: { $mod: [2, 0] } }, { eid: 1, _id: 0 })	Finds employees where the eid is divisible by 2 (even eid values).	[{ eid: 102 }, { eid: 104 }, { eid: 106 }]

MongoDB Array Operators:

Operator	Command	Description	Output
$all	db.employees.find({ skills: { $all: ["Java", "SQL"] } }, { eid: 1, _id: 0 })	Finds employees where the skills array contains **all** the specified elements.	[{ eid: 101 }]
$elemMatch	db.employees.find({ accessid: { $elemMatch: { $gte: 3000, $lt: 5000 } } }, { eid: 1, _id: 0 })	Finds employees where at least one element in the accessid array satisfies the specified condition.	[{ eid: 101 }, { eid: 105 }]
$size	db.employees.find({ skills: { $size: 3 } }, { eid: 1, _id: 0 })	Finds employees where the skills array contains exactly 3 elements.	[{ eid: 101 }, { eid: 102 }, { eid: 103 }, { eid: 105 }, { eid: 106 }]

MongoDB Update Operators:

Operator	Command	Description	Output
$set	db.employees.updateOne({ eid: 101 }, { $set: { salary: 95000 } })	Updates the salary of the employee with eid: 101 to 95000.	{ "acknowledged" : true, "matchedCount" : 1, "modifiedCount" : 1 }
$unset	db.employees.updateOne({ eid: 102 }, { $unset: { address: "" } })	Removes the address field from the employee document with eid: 102.	{ "acknowledged" : true, "matchedCount" : 1, "modifiedCount" : 1 }
$inc	db.employees.updateOne({ eid: 103 }, { $inc: { salary: 5000 } })	Increases the salary of the employee with eid: 103 by 5000.	{ "acknowledged" : true, "matchedCount" : 1, "modifiedCount" : 1 }
$min	db.employees.updateOne({ eid: 104 }, { $min: { salary: 85000 } })	Updates the employee with eid: 104 by setting salary to 85000 if the current salary is higher.	{ "acknowledged" : true, "matchedCount" : 1, "modifiedCount" : 1 }
$max	db.employees.updateOne({ eid: 105 }, { $max: { salary: 90000 } })	Updates the employee with eid: 105 by setting salary to 90000 if the current salary is lower.	{ "acknowledged" : true, "matchedCount" : 1, "modifiedCount" : 1 }
$mul	db.employees.updateOne({ eid: 106 }, { $mul: { salary: 1.1 } })	Multiplies the salary of the employee with eid: 106 by 1.1 (i.e., 10% increase).	{ "acknowledged" : true, "matchedCount" : 1, "modifiedCount" : 1 }
$rename	db.employees.updateOne({ eid: 101 }, { $rename: { "designation": "job_title" } })	Renames the field designation to job_title for the employee with eid: 101.	{ "acknowledged" : true, "matchedCount" : 1, "modifiedCount" : 1 }

Operator	Command	Description	Output
$push	db.employees.updateOne({ eid: 102 }, { $push: { skills: "Node.js" } })	Adds "Node.js" to the skills array of the employee with eid: 102.	{ "acknowledged" : true, "matchedCount" : 1, "modifiedCount" : 1 }
$pull	db.employees.updateOne({ eid: 103 }, { $pull: { skills: "Figma" } })	Removes "Figma" from the skills array of the employee with eid: 103.	{ "acknowledged" : true, "matchedCount" : 1, "modifiedCount" : 1 }

MongoDB Array Update Operators:

Operator	Command	Description	Output
$push	db.employees.updateOne({ eid: 101 }, { $push: { skills: "Python" } })	Adds "Python" to the skills array for employee with eid: 101.	{ "acknowledged" : true, "matchedCount" : 1, "modifiedCount" : 1 }
$push with $each	db.employees.updateOne({ eid: 102 }, { $push: { skills: { $each: ["Docker", "Kubernetes"] } } })	Adds multiple elements ("Docker", "Kubernetes") to the skills array for employee 102.	{ "acknowledged" : true, "matchedCount" : 1, "modifiedCount" : 1 }
$pop	db.employees.updateOne({ eid: 103 }, { $pop: { skills: 1 } })	Removes the **last** element from the skills array for employee with eid: 103.	{ "acknowledged" : true, "matchedCount" : 1, "modifiedCount" : 1 }
$pull	db.employees.updateOne({ eid: 104 }, { $pull: { skills: "Cisco Networks" } })	Removes "Cisco Networks" from the skills array of employee 104.	{ "acknowledged" : true, "matchedCount" : 1, "modifiedCount" : 1 }

Operator	Command	Description	Output
$addToSet	db.employees.updateOne({ eid: 105 }, { $addToSet: { skills: "AWS" } })	Adds "AWS" to the skills array **only if it doesn't already exist** for employee 105.	{ "acknowledged" : true, "matchedCount" : 1, "modifiedCount" : 1 }
$addToSet with $each	db.employees.updateOne({ eid: 106 }, { $addToSet: { skills: { $each: ["DevOps", "Python"] } } })	Adds multiple elements to skills array **only if they don't already exist** for employee 106.	{ "acknowledged" : true, "matchedCount" : 1, "modifiedCount" : 1 }
$pullAll	db.employees.updateOne({ eid: 101 }, { $pullAll: { skills: ["Java", "Spring"] } })	Removes multiple elements ("Java", "Spring") from the skills array for employee 101.	{ "acknowledged" : true, "matchedCount" : 1, "modifiedCount" : 1 }
$push with $sort	db.employees.updateOne({ eid: 102 }, { $push: { skills: { $each: ["HTML", "CSS"], $sort: 1 } } })	Adds elements ("HTML", "CSS") to the skills array and sorts the array in ascending order.	{ "acknowledged" : true, "matchedCount" : 1, "modifiedCount" : 1 }
$push with $slice	db.employees.updateOne({ eid: 103 }, { $push: { accessid: { $each: [9999], $slice: -3 } } })	Adds 9999 to the accessid array and keeps only the last 3 elements.	{ "acknowledged" : true, "matchedCount" : 1, "modifiedCount" : 1 }

MongoDB Aggregation Stage Operators:

Operator	Command	Description	Output
$match	db.employees.aggregate([{ $match: { salary: { $gte: 90000 } } }, { $project: { eid: 1, _id: 0 } }])	Filters documents to match employees where salary >= 90000.	[{ eid: 101 }, { eid: 102 }, { eid: 104 }]

Operator	Command	Description	Output
$group	db.employees.aggregate([{ $group: { _id: "$designation", totalEmployees: { $sum: 1 } } }])	Groups employees by designation and counts the total employees in each group.	[{ _id: "Senior Developer", totalEmployees: 1 }, { _id: "Project Manager", totalEmployees: 1 }]
$project	db.employees.aggregate([{ $project: { eid: 1, _id: 0 } }])	Projects specific fields, in this case, only eid.	[{ eid: 101 }, { eid: 102 }, { eid: 103 }, { eid: 104 }, { eid: 105 }, { eid: 106 }]
$sort	db.employees.aggregate([{ $sort: { salary: -1 } }, { $project: { eid: 1, _id: 0 } }])	Sorts employees by salary in descending order.	[{ eid: 102 }, { eid: 101 }, { eid: 104 }, { eid: 105 }, { eid: 103 }, { eid: 106 }]
$limit	db.employees.aggregate([{ $limit: 3 }, { $project: { eid: 1, _id: 0 } }])	Limits the result to the first 3 employees.	[{ eid: 101 }, { eid: 102 }, { eid: 103 }]
$skip	db.employees.aggregate([{ $skip: 2 }, { $project: { eid: 1, _id: 0 } }])	Skips the first 2 employees and returns the rest.	[{ eid: 103 }, { eid: 104 }, { eid: 105 }, { eid: 106 }]
$unwind	db.employees.aggregate([{ $unwind: "$skills" }, { $match: { skills: "Java" } }, { $project: { eid: 1, _id: 0 } }])	Unwinds the skills array and finds employees with the Java skill.	[{ eid: 101 }]
$addFields	db.employees.aggregate([{ $addFields: { salaryIncrement: { $multiply: ["$salary", 1.10] } } }, { $project: { eid: 1, _id: 0 } }])	Adds a new field salaryIncrement which is 10% more than the current salary, and returns only eid.	[{ eid: 101 }, { eid: 102 }, { eid: 103 }, { eid: 104 }, { eid: 105 }, { eid: 106 }]
$out	db.employees.aggregate([{ $match: { salary: { $gte: 90000 } } }, { $out: "highSalaryEmployees" }])	Writes the result of the pipeline to a new collection called highSalaryEmployees.	Collection highSalaryEmployees will be created with matched documents.

Operator	Command	Description	Output
$count	db.employees.aggregate([{ $match: { salary: { $gte: 90000 } } }, { $count: "high_salary_employees" }])	Counts the number of employees who have a salary greater than or equal to 90000.	[{ high_salary_employees: 3 }]

MongoDB Expression Operators:

Operator	Command	Description	Output
$add	db.employees.aggregate([{ $project: { eid: 1, salaryWithBonus: { $add: ["$salary", 5000] }, _id: 0 } }])	Adds 5000 to the salary of each employee and shows the result in salaryWithBonus.	[{ eid: 101, salaryWithBonus: 95000 }, { eid: 102, salaryWithBonus: 100000 }, ...]
$subtract	db.employees.aggregate([{ $project: { eid: 1, yearsWorked: { $subtract: [{ $year: new Date() }, { $year: "$doj" }] }, _id: 0 } }])	Subtracts the year of doj from the current year to calculate the yearsWorked.	[{ eid: 101, yearsWorked: 14 }, { eid: 102, yearsWorked: 12 }, ...]
$multiply	db.employees.aggregate([{ $project: { eid: 1, salaryIncrease: { $multiply: ["$salary", 1.1] }, _id: 0 } }])	Multiplies the salary by 1.1 to show a 10% salary increase.	[{ eid: 101, salaryIncrease: 99000 }, { eid: 102, salaryIncrease: 104500 }, ...]
$divide	db.employees.aggregate([{ $project: { eid: 1, monthlySalary: { $divide: ["$salary", 12] }, _id: 0 } }])	Divides the salary by 12 to calculate the monthly salary.	[{ eid: 101, monthlySalary: 7500 }, { eid: 102, monthlySalary: 7916.67 }, ...]
$mod	db.employees.aggregate([{ $project: { eid: 1, remainder: { $mod: ["$salary", 1000] }, _id: 0 } }])	Finds the remainder when salary is divided by 1000.	[{ eid: 101, remainder: 0 }, { eid: 102, remainder: 0 }, ...]

Operator	Command	Description	Output
$concat	db.employees.aggregate([{ $project: { eid: 1, fullName: { $concat: ["$firstName", " ", "$lastName"] }, _id: 0 } }])	Concatenates firstName and lastName to form a full name.	[{ eid: 101, fullName: "Arjun Patel" }, { eid: 102, fullName: "Meera Gupta" }, ...]
$ifNull	db.employees.aggregate([{ $project: { eid: 1, primarySkill: { $ifNull: ["$primarySkill", "Not Specified"] }, _id: 0 } }])	Returns the value of primarySkill, or "Not Specified" if primarySkill is null.	[{ eid: 101, primarySkill: "Java" }, { eid: 103, primarySkill: "Not Specified" }, ...]
$cond	db.employees.aggregate([{ $project: { eid: 1, bonusEligible: { $cond: [{ $gte: ["$salary", 95000] }, "Yes", "No"] }, _id: 0 } }])	Conditional operator that checks if salary >= 95000 and returns "Yes" or "No".	[{ eid: 101, bonusEligible: "No" }, { eid: 102, bonusEligible: "Yes" }, ...]
$eq	db.employees.aggregate([{ $project: { eid: 1, isManager: { $eq: ["$designation", "Manager"] }, _id: 0 } }])	Returns true if designation is "Manager", otherwise false.	[{ eid: 101, isManager: false }, { eid: 102, isManager: false }, ...]
$lt	db.employees.aggregate([{ $project: { eid: 1, lowSalary: { $lt: ["$salary", 90000] }, _id: 0 } }])	Returns true if salary is less than 90000.	[{ eid: 103, lowSalary: true }, { eid: 104, lowSalary: false }, ...]
$gte	db.employees.aggregate([{ $project: { eid: 1, highSalary: { $gte: ["$salary", 95000] }, _id: 0 } }])	Returns true if salary is greater than or equal to 95000.	[{ eid: 101, highSalary: false }, { eid: 102, highSalary: true }, ...]
$and	db.employees.aggregate([{ $project: { eid: 1, highEarnAndExperienced: { $and: [{ $gte: ["$salary", 95000] }, { $gte: ["$experience", 10] }] }, _id: 0 } }])	Returns true if both conditions (salary >= 95000 and experience >= 10 years) are true.	[{ eid: 102, highEarnAndExperienced: true }, { eid: 103, highEarnAndExperienced: false }, ...]

Operator	Command	Description	Output
$or	db.employees.aggregate([{ $project: { eid: 1, highEarnOrExperienced: { $or: [{ $gte: ["$salary", 95000] }, { $gte: ["$experience", 10] }] }, _id: 0 } }])	Returns true if either condition (salary >= 95000 or experience >= 10 years) is true.	[{ eid: 102, highEarnOrExperienced: true }, { eid: 103, highEarnOrExperienced: true }, ...]
$type	db.employees.aggregate([{ $project: { eid: 1, salaryType: { $type: "$salary" }, _id: 0 } }])	Returns the BSON data type of the salary field.	[{ eid: 101, salaryType: "int" }, { eid: 102, salaryType: "int" }, ...]
$arrayElemAt	db.employees.aggregate([{ $project: { eid: 1, firstSkill: { $arrayElemAt: ["$skills", 0] }, _id: 0 } }])	Returns the first element of the skills array.	[{ eid: 101, firstSkill: "Java" }, { eid: 102, firstSkill: "Leadership" }, ...]
$concatArrays	db.employees.aggregate([{ $project: { eid: 1, allSkills: { $concatArrays: ["$skills", ["AI", "ML"]] }, _id: 0 } }])	Concatenates the skills array with another array ["AI", "ML"].	[{ eid: 101, allSkills: ["Java", "SQL", "Spring", "AI", "ML"] }, { eid: 102, allSkills: ["Leadership", "AI", "ML"] }]
$size	db.employees.aggregate([{ $project: { eid: 1, skillCount: { $size: "$skills" }, _id: 0 } }])	Returns the number of elements in the skills array.	[{ eid: 101, skillCount: 3 }, { eid: 102, skillCount: 2 }, ...]

MongoDB Array Operations:

Operation	Command	Description	Output
Retrieve with Selection	db.employees.find({ skills: "Java" })	Finds all employees who have "Java" in their skills array.	[{ eid: 101, name: "Arjun Patel", skills: ["Java", "SQL", "Spring"] }]
Retrieve with Projection	db.employees.find({ skills: "Java" }, { eid: 1, name: 1, skills: 1, _id: 0 })	Finds employees with "Java" in skills array and projects only eid, name, and skills.	[{ eid: 101, name: "Arjun Patel", skills: ["Java", "SQL", "Spring"] }]
Update Array Element	db.employees.updateOne({ eid: 101, "skills": "SQL" }, { $set: { "skills.$": "NoSQL" } })	Updates the skills array, replacing "SQL" with "NoSQL" for employee 101.	{ "acknowledged": true, "matchedCount": 1, "modifiedCount": 1 }
Add to Array (Push)	db.employees.updateOne({ eid: 102 }, { $push: { skills: "Node.js" } })	Adds "Node.js" to the skills array for employee 102.	{ "acknowledged": true, "matchedCount": 1, "modifiedCount": 1 }
Add Multiple to Array (Each)	db.employees.updateOne({ eid: 103 }, { $push: { skills: { $each: ["GraphQL", "Docker"] } } })	Adds multiple elements ("GraphQL", "Docker") to the skills array for employee 103.	{ "acknowledged": true, "matchedCount": 1, "modifiedCount": 1 }
Remove from Array (Pull)	db.employees.updateOne({ eid: 104 }, { $pull: { skills: "Cisco Networks" } })	Removes "Cisco Networks" from the skills array for employee 104.	{ "acknowledged": true, "matchedCount": 1, "modifiedCount": 1 }
Remove Multiple from Array (PullAll)	db.employees.updateOne({ eid: 101 }, { $pullAll: { skills: ["Java", "Spring"] } })	Removes multiple elements ("Java", "Spring") from the skills array for employee 101.	{ "acknowledged": true, "matchedCount": 1, "modifiedCount": 1 }

MongoDB Nested Document Field Operations:

Operation	Command	Description	Output
Retrieve with Selection	db.employees.find({ "address.city": "New York" })	Finds all employees who live in "New York" by selecting from the nested address.city field.	[{ eid: 101, name: "Arjun Patel", address: { street: "123 Elm St", city: "New York", state: "NY" } }]
Retrieve with Projection	db.employees.find({ "address.city": "New York" }, { eid: 1, name: 1, "address.city": 1, _id: 0 })	Retrieves employees in "New York" and projects only eid, name, and address.city.	[{ eid: 101, name: "Arjun Patel", address: { city: "New York" } }]
Update Nested Field	db.employees.updateOne({ eid: 101 }, { $set: { "address.city": "San Francisco" } })	Updates the nested address.city field from "New York" to "San Francisco" for employee 101.	{ "acknowledged": true, "matchedCount": 1, "modifiedCount": 1 }
Add Nested Field	db.employees.updateOne({ eid: 102 }, { $set: { "address.zipcode": "94103" } })	Adds a new field zipcode to the nested address document for employee 102.	{ "acknowledged": true, "matchedCount": 1, "modifiedCount": 1 }
Remove Nested Field	db.employees.updateOne({ eid: 103 }, { $unset: { "address.state": "" } })	Removes the address.state field from the nested document for employee 103.	{ "acknowledged": true, "matchedCount": 1, "modifiedCount": 1 }